Please solve all these questions given in the attached PDF

I'll solve all the problems from your AAD (Advanced Algorithms and Design) problem set. This covers Flow Algorithms, Approximation Algorithms, and NP-completeness topics.[1]

Flow Algorithms

Problem 1: Edge Splitting Equivalence

Proof: To show that splitting edge (u, v) yields an equivalent network, we need to prove that the maximum flow value remains unchanged.[1]

Construction: In G', we replace (u, v) with vertex x and edges (u, x) and (x, v) where $c(u, x) = c(x, v) = c(u, v)$.[1]

Forward direction ($G \to G'$): Given a flow f in G, construct flow f' in G' by setting $f'(u, x) = f'(x, v) = f(u, v)$. Since $f(u, v) \leq c(u, v)$ and $c(u, x) = c(x, v) = c(u, v)$, capacity constraints are satisfied. Flow conservation at x holds since incoming flow equals outgoing flow.[1]

Backward direction ($G' \to G$): Given flow f' in G', construct flow f in G by setting $f(u, v) = f'(u, x) = f'(x, v)$. By flow conservation at x, $f'(u, x) = f'(x, v)$, so this is well-defined. Since the bottleneck capacity in the path through x is $\min(c(u, x), c(x, v)) = c(u, v)$, capacity constraints are satisfied.[1]

Problem 2: Multiple Source-Sink Extension

Construction: For a multiple-source multiple-sink network with sources $S = \{s_1, s_2, \ldots, s_k\}$ and sinks $T = \{t_1, t_2, \ldots, t_m\}$, create a supersource s and supersink t.[1]

Add edges (s, s_i) with capacity $c(s, s_i) = \infty$ for all i, and edges (t_j, t) with capacity $c(t_j, t) = \infty$ for all j.[1]

Forward correspondence: Any flow in the multi-source/sink network can be extended to the single-source/sink network by routing all flow from sources through s and all flow to sinks through t. Since supersource/sink edges have infinite capacity, they never become bottlenecks.[1]

Backward correspondence: Any flow in the single-source/sink network restricts to a valid flow in the original network by removing s and t. Flow conservation is maintained at all original vertices.[1]

Problem 3: Vertex Capacities Transformation

Construction: For each vertex $v \in V$ with capacity $l(v)$, split it into two vertices v_{in} and v_{out} connected by edge (v_{in}, v_{out}) with capacity $c(v_{in}, v_{out}) = l(v)$.[1]

For every edge $(u, v) \in E$, replace it with edge (u_{out}, v_{in}) with the same capacity. Source and sink vertices are not split (or have infinite capacity on their connecting edge).[1]

Correctness: The flow through (v_{in}, v_{out}) is bounded by $l(v)$, which enforces the vertex capacity constraint. All other flow properties are preserved.[1]

Complexity: G' has $|V'| = 2|V|$ vertices and $|E'| = |E| + |V|$ edges [1].

Problem 4: Ford-Fulkerson Execution

Without seeing Fig-4.1 in the PDF, I'll describe the general execution steps :[1]

1. Initialize flow $f(e) = 0$ for all edges
2. Construct residual graph G_f
3. Find an augmenting path from source to sink using BFS or DFS
4. Compute bottleneck capacity along the path
5. Augment flow along the path
6. Update residual graph
7. Repeat until no augmenting path exists

The algorithm terminates when no path exists from source to sink in the residual graph.[1]

Problem 5: Maximum |E| Augmenting Paths

Algorithm: First, find the maximum flow using any max-flow algorithm. Then, decompose the flow into at most $|E|$ flow paths [1].

Proof: Any flow can be decomposed into flow paths and cycles. Since cycles don't contribute to net flow from source to sink, we only need paths. Each path uses at least one edge, and each edge can appear in at most one path in the decomposition (we saturate and remove edges as we extract paths). Therefore, the number of paths is at most $|E|$ [1].

Procedure: Start with the maximum flow. Repeatedly find a path from s to t with positive flow, determine the minimum flow value on that path, subtract it from all edges, and record the path. Continue until no positive flow remains.[1]

Problem 6: Edge Connectivity

Algorithm: To find edge connectivity k of undirected graph $G = (V, E)$:[1]

For each pair of vertices (s, t) where $s \neq t$, create a flow network by:

1. Directing each undirected edge in both directions with capacity 1

2. Computing max-flow from s to t
3. By max-flow min-cut theorem, this gives the minimum number of edges whose removal disconnects s and t

The edge connectivity is $k = \min_{s \neq t} \text{maxflow}(s, t)$.[1]

Optimization: We only need to run max-flow from a single vertex s to all other vertices $t \in V \setminus \{s\}$, giving $|V| - 1$ max-flow computations [1].

Each flow network has $O(V)$ vertices and $O(E)$ edges (since each undirected edge becomes two directed edges).[1]

Problem 7: Dynamic Max-Flow Updates

(a) Increasing capacity: When $c(u, v)$ increases by 1, check if there's an augmenting path from s to t in the residual graph that uses edge (u, v).[1]

Run a single BFS/DFS from s to t in the updated residual graph. If a path exists, augment by 1 unit. Time complexity: $O(V + E)$ for one BFS/DFS.[1]

(b) Decreasing capacity: When $c(u, v)$ decreases by 1, if current flow $f(u, v) \leq c(u, v) - 1$, no change needed.[1]

If $f(u, v) = c(u, v)$ (edge was saturated), reduce flow by 1 on (u, v), and search for an augmenting path from u to v in the residual graph to reroute this flow. Use BFS/DFS for $O(V + E)$ time.[1]

Approximation Algorithms

Problem 8: 2-Approximation for Vertex Cover

Suboptimal example: Consider a triangle graph K_3 with vertices $\{a, b, c\}$ and edges $\{(a, b), (b, c), (a, c)\}$.[1]

The 2-approximation algorithm might pick edges (a, b) and (b, c), yielding vertex cover $\{a, b, c\}$ with size 3. However, the optimal vertex cover is any 2 vertices, e.g., $\{a, b\}$, with size 2.[1]

Optimal example: Consider a star graph with center c and k leaf vertices. The 2-approximation algorithm picks one edge, say (c, v_1), adding both c and v_1 to the cover. Then c covers all remaining edges.[1]

The algorithm yields size 2, and the optimal is size 1 (just c). However, if the matching happens to pick multiple edges not sharing vertices (which won't happen in a star), the approximation can be optimal in graphs with perfect matchings where the matching itself is the minimum vertex cover.[1]

Problem 9: Maximal Matching Property

Proof: Let M be the set of edges picked by the 2-approximation algorithm.[1]

By construction, no two edges in M share a vertex (the algorithm removes both endpoints after picking an edge). Therefore, M is a matching.[1]

To show maximality, suppose there exists an edge $(u, v) \notin M$ that could be added to M. This means neither u nor v is covered by any edge in M. But the algorithm continues until no uncovered edges remain, contradiction. Therefore, M is a maximal matching.[1]

Problem 10: Optimal Vertex Cover for Trees

Greedy Algorithm: Process the tree bottom-up (post-order traversal) :[1]

1. Root the tree at any vertex
2. For each leaf vertex v, if v is not in the cover, add its parent to the cover
3. Remove covered edges and continue upward

Linear-time algorithm: Use dynamic programming :[1]

- For each vertex v, compute $DP[v]$ = minimum cover size for subtree rooted at v when v is NOT included
- $DP[v]$ = minimum cover size when v IS included[1]

Recurrence:

- $DP[v] = \sum_{u \in \text{children}(v)} DP[u]$ (if v not included, all children must be)[1]
- $DP[v] = 1 + \sum_{u \in \text{children}(v)} \min(DP[u], DP[u])$[1]

Time complexity: $O(V)$ as each vertex is processed once.[1]

Problem 11: Clique Approximation Impossibility

Answer: No, this does not imply a polynomial-time constant-factor approximation for clique.[1]

Reasoning: While vertex cover has a 2-approximation, the complementary relationship doesn't preserve approximation ratios. A ρ-approximation for vertex cover on G doesn't translate to a ρ -approximation for clique on $\overline{G}$.[1]

Specifically, if the optimal vertex cover has size k and the optimal clique in $\overline{G}$ has size $n - k$, a 2-approximation giving cover size $2k$ corresponds to a clique of size at most $n - 2k$ in the complement. The ratio $\frac{n-k}{n-2k}$ is unbounded as k approaches $n/2$.[1]

This explains why clique is hard to approximate (no constant-factor approximation unless P=NP) despite vertex cover having a 2-approximation.[1]

Problem 12: Non-negative Costs with Triangle Inequality

Proof: Given triangle inequality $c(u,v) \leq c(u,w) + c(w,v)$ for all vertices u, v, w.[1]

For any vertices u, v, consider a third vertex w (exists since $|V| \geq 3$). Apply triangle inequality:

- $c(u,v) \leq c(u,w) + c(w,v)$
- $c(u,w) \leq c(u,v) + c(v,w)$
- $c(w,v) \leq c(w,u) + c(u,v)$

From the second inequality: $c(u,w) - c(u,v) \leq c(v,w)$

From the third inequality: $c(w,v) - c(u,v) \leq c(w,u) = c(u,w)$

Adding the first inequality with $w = v$ (considering the edge as a degenerate path): $c(u,v) \leq c(u,v) + c(v,v)$, which implies $c(v,v) \geq 0$. By setting $w = u$ in the triangle inequality: $c(u,v) \leq c(u,u) + c(u,v)$, so $c(u,u) \geq 0$.[1]

For distinct u, v, using vertices u, v, w: $c(u,w) \leq c(u,v) + c(v,w)$ and $c(v,w) \leq c(v,u) + c(u,w)$. If $c(u,v) < 0$, we could construct a contradiction by choosing appropriate w, but more directly: since this is a complete graph with triangle inequality, and $c(v,v) = 0$, we have $c(u,v) \leq c(u,v) + c(v,v) = c(u,v) + 0$, and symmetrically $c(u,u) = 0$. Using $c(u,v) \leq c(u,u) + c(u,v) = c(u,v)$ and $0 = c(u,u) \leq c(u,v) + c(v,u) = 2c(u,v)$, we get $c(u,v) \geq 0$.[1]

Problem 13: TSP Transformation

Construction: Given general TSP instance $(G = (V,E), c)$, create new instance (K_n, c') where K_n is complete graph on V :[1]

$$c'(u,v) = \begin{cases} c(u,v) & \text{if } (u,v) \in E \\ M & \text{if } (u,v) \notin E \end{cases}$$

where \$$M = 1 + \sum_{(u,v) \in E} c(u,v)$ (larger than any possible tour in original graph).[1]

Correctness: Both instances have the same optimal tours because any tour using an edge not in E has cost $\geq M$, which exceeds any tour using only edges in E. Therefore, optimal tours are identical.[1]

Why no contradiction: This transformation doesn't contradict the inapproximability of general TSP because the transformation itself depends on the solution structure. While we can transform the instance in polynomial time, an approximation algorithm for metric TSP cannot be applied to solve general TSP, because the instances we create have very large edge weights that make the approximation ratio meaningless (approximation ratio becomes M-dependent, not constant).[1]

NP-Completeness and Reductions

Problem 14: Complement Reduction Equivalence

Proof: We need to show $L \leq_p \overline{L}$ if and only if $\overline{L} \leq_p L$.[1]

Forward direction: Assume $L \leq_p \overline{L}$ via reduction f. Then $x \in L \iff f(x) \in \overline{L} \iff f(x) \notin L$.[1]

To show $\overline{L} \leq_p L$, we use the same function f. For $x \in \overline{L}$: $x \in \overline{L} \iff x \notin L \iff f(x) \notin \overline{L} \iff f(x) \in L$. Therefore, f reduces $\overline{L}$ to L.[1]

Backward direction: Symmetric argument. If $\overline{L} \leq_p L$ via g, then $x \in \overline{L} \iff g(x) \in L$. For $x \in L$: $x \in L \iff x \notin \overline{L} \iff g(x) \notin L \iff g(x) \in \overline{L}$. Therefore, g reduces L to $\overline{L}$.[1]

Problem 15: DNF-SAT is in P

Algorithm: A formula in Disjunctive Normal Form (DNF) is a disjunction of conjunctive clauses: $(C_1) \vee (C_2) \vee \ldots \vee (C_k)$ where each C_i is a conjunction of literals.[1]

The formula is satisfiable if and only if at least one clause C_i is satisfiable. A conjunctive clause is satisfiable if and only if it doesn't contain both a variable and its negation.[1]

Polynomial-time algorithm:

1. For each clause C_i, check if it contains both x and $\neg x$ for any variable x
2. If any clause has no contradictions, return SATISFIABLE
3. If all clauses have contradictions, return UNSATISFIABLE

Time complexity: $O(n \cdot m)$ where n is the number of clauses and m is the maximum clause size.[1]

Problem 16: Complexity Classes

Definitions :[1]

P: The class of decision problems solvable by a deterministic Turing machine in polynomial time.[1]
Example: Sorting, shortest path, minimum spanning tree.[1]

NP: The class of decision problems verifiable by a deterministic Turing machine in polynomial time (i.e., there exists a polynomial-time algorithm to verify a given certificate).[1]
Example: SAT, Hamiltonian path, vertex cover.[1]

co-NP: The class of decision problems whose complement is in NP.[1]
Example: Tautology (complement of SAT), non-Hamiltonicity.[1]

NP-hard: The class of problems at least as hard as the hardest problems in NP. Formally, a problem H is NP-hard if every problem in NP can be reduced to H in polynomial time.[1]
Example: Halting problem, general TSP.[1]

NP-complete: The class of decision problems that are both in NP and NP-hard.[1]
Example: SAT, 3SAT, clique, vertex cover, Hamiltonian cycle.[1]

Problem 17: 0-1 Knapsack Complexity

Analysis: The dynamic programming algorithm for 0-1 knapsack runs in $O(nW)$ time, where n is the number of items and W is the knapsack capacity.[1]

Answer: This is NOT a polynomial-time algorithm in the input size.[1]

Proof: The input size is $O(n \log W + n \log v_{max})$ where v_{max} is the maximum value. The number W requires only $\log W$ bits to represent. The running time $O(nW)$ is exponential in the input size because $W = 2^{\log W}$, making the algorithm pseudo-polynomial (polynomial in the numeric value of the input, but exponential in the input size).[1]

Polynomial restriction: If we restrict W to be polynomial in n, say $W = O(n^k)$ for some constant k, then $O(nW) = O(n^{k+1})$, which is polynomial in n. In this case, the input size is $O(n \log n) = O(n \log n)$, and the algorithm becomes polynomial in the input size.[1]

Problem 18: DOUBLE-SAT is NP-Complete

Proof :[1]

Step 1 - DOUBLE-SAT ∈ NP: Given formula ϕ and two satisfying assignments a_1 and a_2, verify in polynomial time that both satisfy ϕ and that $a_1 \neq a_2$. This can be done in $O(|\phi|)$ time [1].

Step 2 - DOUBLE-SAT is NP-hard: Reduce SAT to DOUBLE-SAT.[1]

Given formula ϕ on variables $x_1, \ldots, x_n$, construct $\phi' = \phi \wedge (y \vee \neg y)$ where y is a new variable not in ϕ.[1]

Correctness: If ϕ is satisfiable with assignment a, then ϕ' has at least two satisfying assignments: $(a, y = 0)$ and $(a, y = 1)$. If ϕ is unsatisfiable, then ϕ' is unsatisfiable.[1]

Therefore, $\phi \in \text{SAT} \iff \phi' \in \text{DOUBLE-SAT}$. The reduction is polynomial-time.[1]

Problem 19: 3SAT ≤p CLIQUE

Reduction: Given 3SAT formula $\phi = C_1 \wedge C_2 \wedge \ldots \wedge C_k$ where each C_i has 3 literals.[1]

Construction: Build graph $G = (V, E)$:[1]

- For each clause $C_i = (l_{i1} \vee l_{i2} \vee l_{i3})$, create 3 vertices v_{i1}, v_{i2}, v_{i3} corresponding to the literals
- Add edge (v_{ij}, v_{pq}) if and only if $i \neq p$ (different clauses) AND l_{ij} is not the negation of l_{pq}

Output (G, k) where k is the number of clauses.[1]

Correctness:

- **Forward:** If ϕ is satisfiable with assignment a, select one true literal from each clause. The corresponding k vertices form a k-clique because they're from different clauses and no two are contradictory (since a satisfies both).[1]
- **Backward:** If G has a k-clique, it contains exactly one vertex from each clause (since $|V| = 3k$ and edges only exist between different clauses). Set variables to make these literals true. No conflicts arise because clique edges ensure no contradictory literals are chosen [1].

Problem 20: 3SAT ≤p VERTEX-COVER

Reduction: Given 3SAT formula $\phi = C_1 \wedge C_2 \wedge \ldots \wedge C_m$ with variables $x_1, \ldots, x_n$.[1]

Construction of graph $G = (V, E)$:[1]

1. **Variable gadgets:** For each variable x_i, create two vertices v_i and $\overline{v_i}$ connected by edge $(v_i, \overline{v_i})$
2. **Clause gadgets:** For each clause $C_j = (l_{j1} \vee l_{j2} \vee l_{j3})$, create a triangle with vertices c_{j1}, c_{j2}, c_{j3}
3. **Connector edges:** For each literal l_{jp} in clause C_j, add edge from c_{jp} to the corresponding variable vertex (e.g., if $l_{jp} = x_i$, connect to v_i; if $l_{jp} = \neg x_i$, connect to $\overline{v_i}$)

Set $k = n + 2m$.[1]

Correctness:

- **Forward:** If ϕ is satisfiable with assignment a, construct vertex cover: (1) For each variable gadget, pick the vertex corresponding to false literal (covers variable edges), (2) For each clause triangle, the assignment makes at least one literal true, so that clause vertex connects to a selected variable vertex; pick the other 2 triangle vertices (covers triangle edges and connector edges).[1]
- **Backward:** If G has a k-vertex cover, it must include: (1) exactly one vertex from each variable gadget (need n vertices to cover n variable edges), (2) exactly 2 vertices from each clause triangle (need at least 2 per triangle to cover 3 edges, totaling $2m$ vertices). The uncovered vertex in each triangle connects to a variable vertex not in the cover, meaning that literal is true, satisfying the clause.[1]

Problem 21: Hamiltonian Path is NP-Complete

Proof :[1]

Step 1 - HAM-PATH ∈ NP: Given graph G and a path p, verify in polynomial time that p visits each vertex exactly once.[1]

Step 2 - Reduction: Reduce from Hamiltonian Cycle (known NP-complete).[1]

Given graph $G = (V, E)$ for HAM-CYCLE, select arbitrary vertex $v \in V$. Create $G' = (V', E')$:[1]

- $V' = V \cup \{s, t\}$ (add two new vertices)

- $E' = E \cup \{(s, u) : (v, u) \in E\} \cup \{(u, t) : (u, v) \in E\}$

Correctness: G has Hamiltonian cycle if and only if G' has Hamiltonian path from s to t.[1]

- **Forward:** If G has cycle $v \to v_1 \to \ldots \to v_k \to v$, then G' has path $s \to v_1 \to \ldots \to v_k \to t$
- **Backward:** If G' has Ham-path from s to t, it must start with $s \to u$ where $(v, u) \in E$ and end with $w \to t$ where $(w, v) \in E$. The path in G' corresponds to cycle $v \to u \to \ldots \to w \to v$ in G

Problem 22: LPATH is NP-Complete

Proof :[1]

Step 1 - LPATH ∈ NP: Given path p of length $\geq k$ from a to b, verify in polynomial time that it's a simple path.[1]

Step 2 - Reduction: Reduce Hamiltonian Path to LPATH.[1]

Given graph $G = (V, E)$ for HAM-PATH with designated start s and end t, construct instance $(G, s, t, |V| - 1)$ for LPATH [1].

Correctness: A Hamiltonian path has length exactly $|V| - 1$ (visits all $|V|$ vertices using $|V| - 1$ edges). Therefore, G has Ham-path from s to t if and only if G has simple path of length $\geq |V| - 1$ from s to t [1].

Since simple paths have length at most $|V| - 1$, the conditions are equivalent [1].

Problem 23: Set Cover is NP-Complete

Reduction: Reduce VERTEX-COVER to SET-COVER.[1]

Given graph $G = (V, E)$ and integer k for VERTEX-COVER.[1]

Construction for SET-COVER:

- Universe $U = E$ (set of all edges)
- Family of sets $\mathcal{F} = \{S_v : v \in V\}$ where $S_v = \{e \in E : v \text{ is incident to } e\}$
- Budget $k' = k$

Correctness: A vertex cover of size k corresponds to k sets in $\mathcal{F}$ whose union is E, and vice versa.[1]

- **Forward:** If $C \subseteq V$ is a vertex cover of size k, then $\bigcup_{v \in C} S_v = E$ because every edge is incident to at least one vertex in C
- **Backward:** If sets $\{S_{v1}, \ldots, S_{vk}\}$ cover $U = E$, then $\{v_1, \ldots, v_k\}$ is a vertex cover of size k because every edge is in some S_{vi}, meaning it's incident to v_i

The reduction is polynomial-time: $O(|V| + |E|)$ [1].

⁂

1. AAD_QB_1_5-All-Topics.pdf